Step 1- Review Inputs:

Constraints, Use cases, Concerns, Quality Attributes of AIDAP:

Use cases, quality attributes, Primary Attributes:

Use Case	Description
UC-1 Query Academic Information	Always allows a student to ask academic inquiries, such as when their next exam is. The system will then understand the question using Natural Language Processing. It will further output the date and time from the school.
UC-3 Personalized Dashboard Access	User can access a personalized dashboard that provides a summary for upcoming events, as well as for academic performance.
UC-4 Course Announcement Management	Lecturer shall be able to post announcements by text or voice to their students.
UC-5 Management of Integrations	Administrators should be allowed to manage the connections the system has to external systems such as LMS, calendar, registration as well as email systems.
UC-7 System Health Monitor	System maintainer shall be able to view the monitoring dashboards showing latency, error rate and health of the system.
UC-8 AI model updates	The maintainer shall be able to put out new AI models and API key configurations without downtime.

ID	Quality Attribute	Scenario	Associated Use Case
QA-1	Performance	95% of student queries are answered in less than 2 seconds with normal load.	UC-1
QA-2	Availability	The system should have a 99.5 percent uptime monthly, with automatic failover and backup recovery	UC-2, UC-7
QA-3	Scalability	The system should handle up to a peak of 5000 concurrent users while maintaining a response time of less than 2 seconds.	UC-1, UC-5
QA-4	Security	The system should have role-based access and is authenticated and documented using Single Sign On, not allowing unauthorized access.	UC-6
QA-5	Modifiability	New AI models can be integrated into the system without updating core components, with zero downtime deployment	UC-8

Consider priority

System Constraints:

ID	Constraint
CON-1	System to follow accepted guidelines for privacy as per university's rules
CON-2	The system must be able to run on cloud servers while being able to handle up to 5000 users.
CON-3	The system must maintain an average of less than 2 seconds of response time under normal load.
CON-4	Must allow updates without any downtime using pipelines for continuous deployment.
CON-5	The system recovers from failure without loss of stored data for any user.

System Concerns:

ID	Concern
CRN-1	Ensuring seamless integration between multiple systems from the institute (LMS, registration, calendar, email) that use different APIs and data formats.
CRN-2	Maintaining data synchronization and consistency across the distributed sources, and not having any performance drops.
CRN-3	Guaranteeing secure handling and storage of sensitive academic and personal information and also complying with privacy laws.
CRN-4	How will continuous integration and deployment be configured to allow zero-downtime updates?
CRN-5	Potential difficulty in accessing real-time health and error metrics without compromising operational security?

START OF DOC

Step 1 - Review Inputs:

Category	Details
Design Purpose	This is a greenfield system from a mature domain. The system is designed to deliver a unified AI assistant that provides fast, personalized access to university data and services through conversational interaction.
Primary Functional Requirements	UC-1: Directly supports core business logic, as students need to submit academic queries. UC-4: Directly supports core business logic, allowing teachers to submit academic information. UC-8: Directly supports core AI business logic, allowing for maintainability.
Quality Attribute Scenarios	Prioritized Scenario listed below:

<table>
<tr><td></td><td>
<table>
<tr><th>Scenario ID</th><th>Importance to the Customer</th><th>Difficulty of Implementation According to the Architect</th></tr>
<tr><td>QA-1</td><td>High</td><td>High</td></tr>
<tr><td>QA-2</td><td>High</td><td>High</td></tr>
<tr><td>QA-3</td><td>Medium</td><td>Medium</td></tr>
<tr><td>QA-4</td><td>High</td><td>Low</td></tr>
<tr><td>QA-5</td><td>Medium</td><td>High</td></tr>
</table>
Based on this, QA-1, QA-2, QA-4 are selected as drivers.
</td></tr>
<tr><td>Constraints</td><td>CON-1: Pivotal to follow rules of the school.
CON-2: Having alot of users is crucial for the system.
CON-5: Minimal downtime allows for frequent queries, which is very important.</td></tr>
<tr><td>Architectural Concerns</td><td>CRN-1: Seamless integration is pivotal for business case, allows for easy querying of data from different sources.
CRN-3: Keeping academic information safe is detrimental to the system.</td></tr>
</table>

Iteration 1 - Establishing a general system view

Step 2 - Establishing goals by selecting drivers

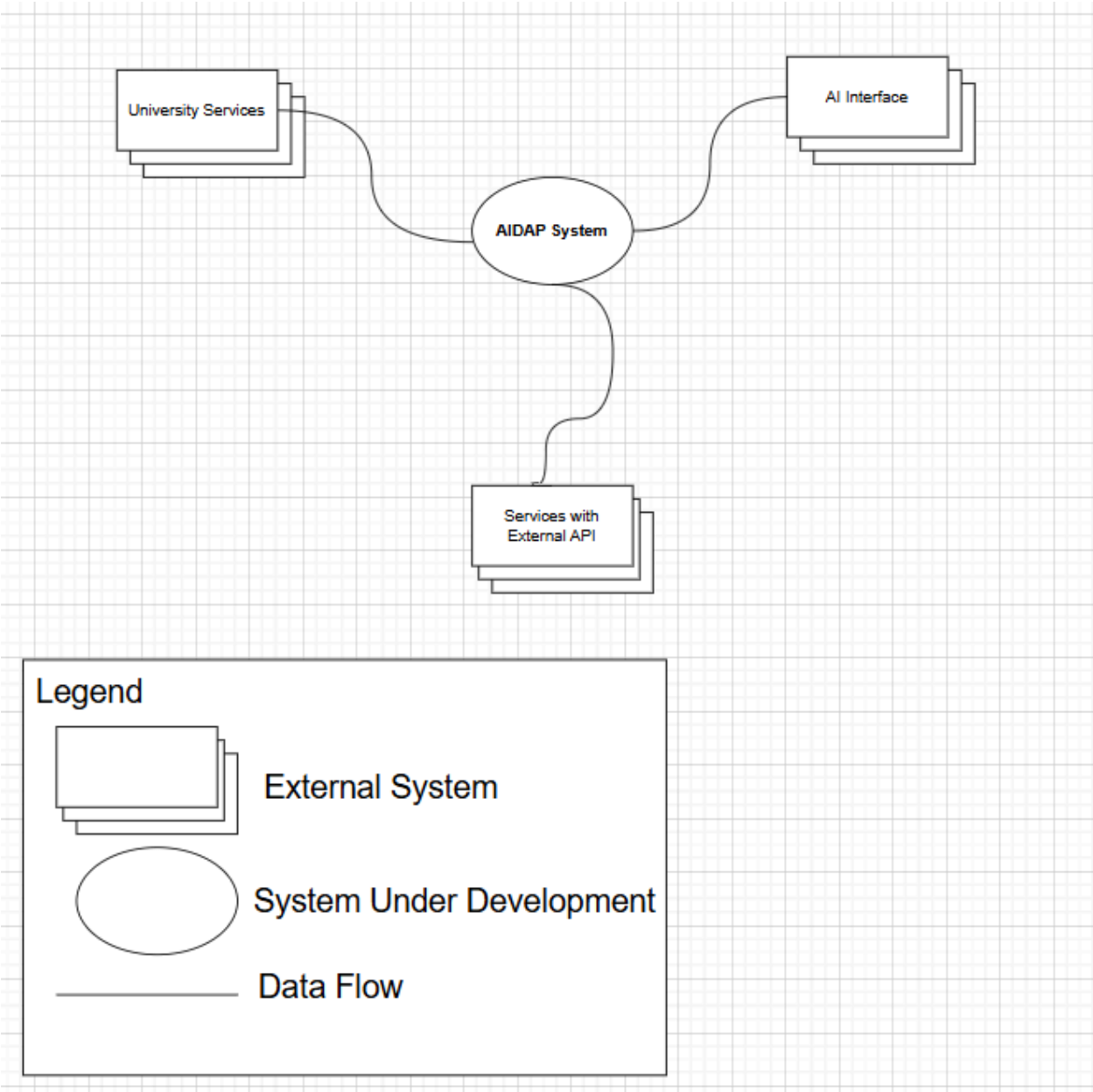
This is the first iteration in the design of a greenfield system, so the iteration goal is to create a general system structure. The architect for the system will be mindful of the following drivers:
QA-1: Performance
QA-2: Availability
QA-4: Security

CON-1: Pivotal to follow rules of the school.
CRN-1: External integration to school system
CRN-3: Keeping academic information safe is detrimental to the system

Each of these use cases described in the bottom table:

Step 3 - Choose an element to decompose

Since this is a greenfield system in a mature domain, the element we need to redefine is the entire AIDAP system. The diagram below shows the Content Diagram for the system.



Step 4 - Choose one or more Design Concepts:

Design Decision and Location	**Rationale**
Will structure the client part of the system using the **Web Application** reference architecture.	The Web Application reference architecture supports the development of applications that communicate through a server using HTTP Protocol. This is crucial to our system, as we need to have a system that is accessible through every device (QA-2) and being

	accessible through the web is an easy way to do that. This will allow us to use HTML elements to expertly develop a personal dashboard (UC-2). The web app reference architecture uses three-tier approach for the logic, and it allows us to relate a database to the logic of the system (QA-4). We rejected **Rich Internet Applications** as when we looked into it, it relies heavily on outdated technology such as Silverlight. Since we want software that doesn't need much maintenance, as wellwe went with the Web Application reference architecture. We rejected **Rich Client Applications** as we need our application to be accessible for every user, and installing everything onto a local users device would result in a very bloated software without internet connectivity, especially with the AI interface. We rejected **Mobile Applications** as this means the user cannot access the platform on any device, violating the need to have a portable program. We rejected **Service Applications** as the user needs to interact with the platform, directly conflicting with the need to have an interactive program that queries information.
Will implement the System's core behavior the **Microservices** Reference Architecture	The **Microservices** Reference Architecture supports the system's need for scalability to 5000 users (CON-2), Zero-downtime deployments(QA-5), as well as AI Integration(UC-8). We rejected **Layered (N-Tier) Architecture** as while the structure provides clean separation, it lacks flexibility for scaling and continuous deployment, which is essential for

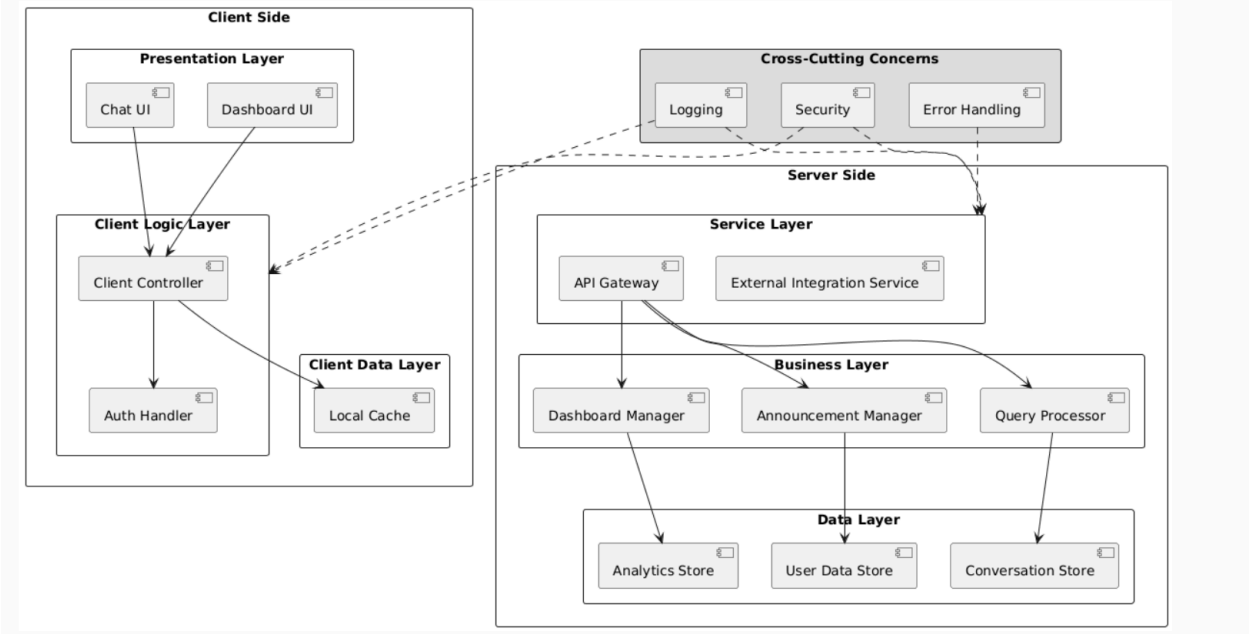
	our platform. We rejected **Event-driven Architecture** as, upon further research, they dont support synchronous interactions requires for answering queries within 2 seconds, going against our business logic. **Monolithic Architecture** was rejected, as in order to make changes to our AI, for example, the entire system would have to be redeployed, making the system incur downtimes. **Serverless Architecture** was rejected because upon further research, they have cold-start delays that conflict with our systems performance requirements, and handling 5000 users at once would be very inconsistent, as it isn't suited for that.
Will implement the Physical Structure of the application using the **Three-tier deployment pattern**.	Since the system must be accessed from anywhere (QA-2), and we need to use a Database to store school data, a three-tier deployment pattern is appropriate. This will allow us to properly support querying of information (UC-1), and integration management (UC-5) by separating the interface, processing logic of the system, and institutional data access into different tiers. This allows us to also easily support scaling up to 5000 users (QA-3). We rejected **Non-Distributed Deployment** because upon further research, they don't have the means to meet system's requited performance and availability targets, like scaling up to 5000 users. We rejected **Distributed Deployement** because the structure doesn't support our system's low-latency, predictable interactions, and would make data security difficult across different nodes.

Define ref architecture for client and server if needed, specify alternatives for other ref architectures why we didn't select. Reference for deployment pattern, reference logic and deployment patterns, add framework ex django for security and privacy

Step 5 - Instantiate Architectural elements, allocate responsibilities and define interfaces

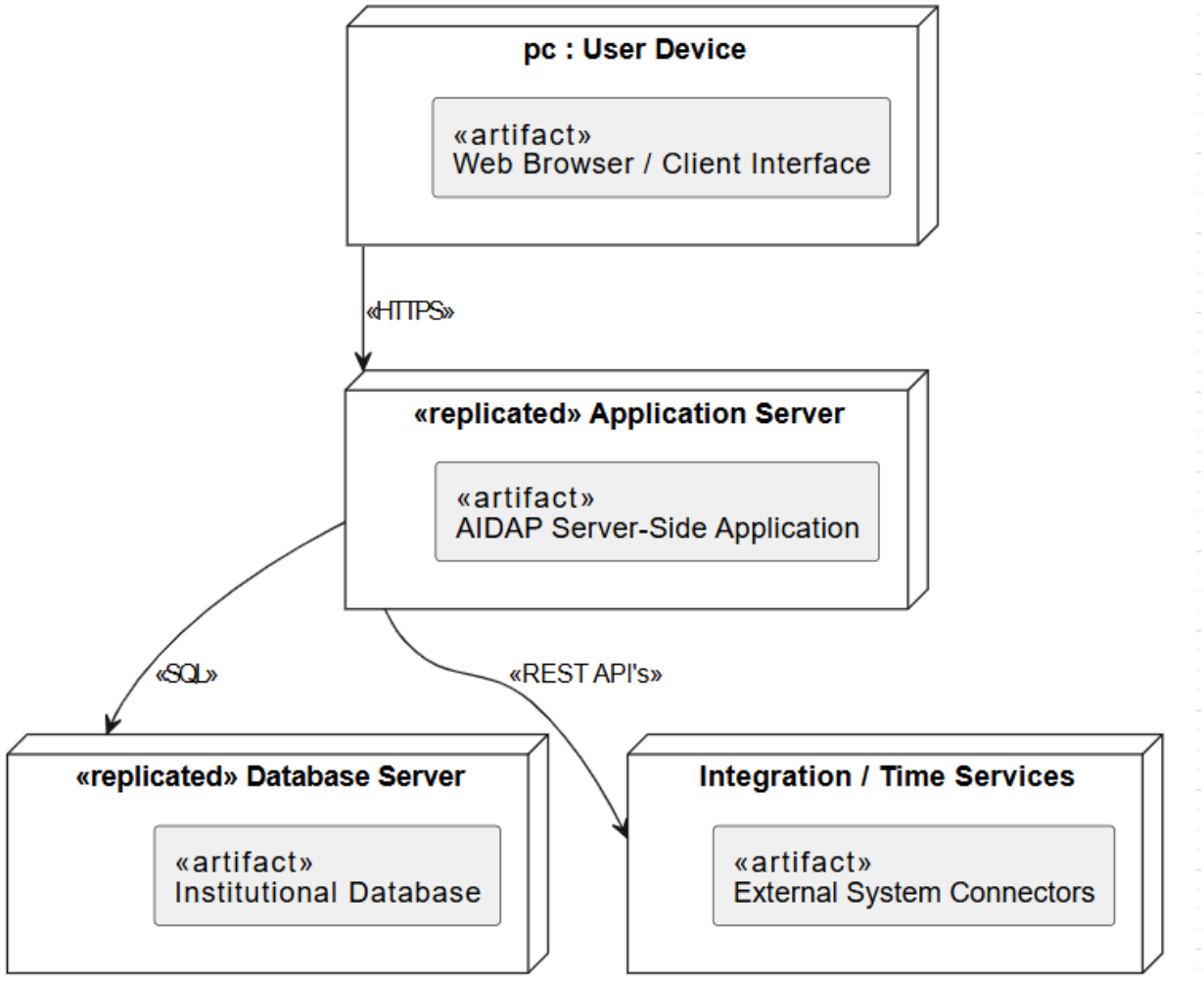
Design Decision & Location	Rationale
Use the web application reference Architecture [client layer]	Ensures the system is accessible from any device via a browser, supporting portability. Also supports a clean three-tier structure that integrates well with backend logic and databases.
Use the Microservices Reference Architecture [Server-Side Core Logic]	Provide independent scalability, supporting up to 5000 users. Also enables zero-downtime deployments through component-level upgrades.
Use the Three-Tier Deployment Pattern[physical Structure of the Application]	Supports universal access from any device. Also enables reliable querying of institutional data and integration management. Allows independent Scaling of tiers to meet the required concurrency
Implement an API Gateway as the Single Entry Point for all clients	Enables standardized SSO token validation, rate-limiting, traffic routing, and TLS termination, which improves both security and performance. Allows centralized monitoring of requests metrics, essential for meeting QA-1 and QA-3

Step 6 - Sketch Views and Record Design Decisions



Element	Responsibility
Chat UI	Displays the conversational interface, sends both staff and student questions to the backend, and displays the answers (UC-1)
Dashboard UI	Displays personalized dashboard that includes deadlines, academic information, and events pertinent to the user that is logged in (UC-3)
Client Controller	Screen flow is managed between both the chat and the dashboard, and forwarding of user actions from the UI to the API Gateway
Auth Handler	Start single sign on login, tracks user that is logged in, and connects identity information to client requests (UC-6)
Local Cache	Stores recent data temporarily on the client side to help improve performance, as well as help with limited connectivity
Logging	Records events and requests throughout the system that is important to allow behaviour to be analyzed and reviewed later (UC-7)

Security	Security checks that include academic data protection and access control.
Error Handling	Finds out and handles failures in a way that returns error information to either user or maintainer.
API Gateway	Serves as the main gateway for client requests, performing initial checks and directing each requests to the correct backend service
External Integration Service	Handles connections to the institutional platforms like the LMS, registration system, calendar, and emails ensuring that AIDAP’s internal data remains synchronized with the systems
Query Processor	Understands User questions, works with AI components and data sources, and returns reponses for academic or scheduling inquiries
Dashboard Manager	Gathers and aggregates the information required for a user's dashboard, including upcoming tasks, assignments and events.
Announcement Manager	Enables instructors and administrators to create and manage course or campus announcements that students can view and receive notifications for
User Data Store	Maintains user profiles, roles and preferences that are required for authentication and personalized behaviour
Conversation Store	Stores a history of interactions between the user and the assistant so the system can offer context-aware and personalized responses while still following privacy rules
Analytics Store	Saves combined usage and performance data that is used for system monitoring and generating reports.



Relationship	**Description**
Between User Device and Application Server	HTTPS is used to perform communication between the two, Client requests sent to API Gateway within AIDAP Server-Side Application.
Between Application Server And Database	Server has communication to the SQL Database.
Between Application Server and Integration/Time Services	Communcation is done through REST API's

Step 7 - Perform analysis of current design and review iteration goal and design objectives

The goal of iteration 1 was to create a system structure for AIDAP using the selected design concepts.

-Web application reference architecture for client system access, Microservices reference architecture for the server side logic, and a Three tier deployment pattern for physical.

We now evaluate how effective this first design is at meeting the identified architectural drivers.

Driver	**Status**	**How it is addressed**
QA-1 Performace	Partial Address	The microservices reference architecture and the Three-tier deployment pattern enable horizontal scaling and clear separation of concerns, which helps maintain query responses to under 2 seconds. However, we have not chosen specific tactics as a result, this quality requirement is partly met in the current iteration
QA-2 Availability	Partial Address	The three-tier deployment pattern and microservices allow services to be replicated and deployed across multiple nodes that support automatic failovers and backup recoveries. Certain details, such as health-check mechanisms, redundancy levels and failover workflows, need to be refined in later iterations
QA-4 Security	Partial Address	Centralizing logic on the server makes it easier to enforce security controls and SSO at the server side, supporting role-based access. Specific security framework, encryption mechanisms, and audit logging policies have yet to be chosen; as a result, the scenario is only partially covered.
CON-1 University Privacy Rules	Partial Address	Keeping all private academic data on the server and separating from the presentation and client tiers ensures compliance with privacy rules. Concrete data retention policies and access control still need to be specified
CON-2 Cloud Deployment and 5000 users	Partial Address	The Microservices and Three-tier architectures are compatible with cloud platforms and support scaling individual services to manage heavy loads. However, detailed capacity planning decisions and various deployment topologies are only partially complete
CON-5 Recovery without data loss	Not fully addressed	The design assumes a database and deployment pattern is capable of supporting backup and restore

		functionality. The exact mechanisms have not been specified. This will be addressed in a future release.
CRN-1 Seamless Integration with LMS/registration/calendat/email	Partial Address	The microservices approach allows future integration with external systems via dedicated services. Though, the exact integration methods still need to be determined.
CRN-3 Secure handling and storage of academic information	Partial Address	Using centralized server-side logic and a clear separation of the data tier provides a solid base for securely storing academic information. More detailed security measures will be introduced later.

Overall, iteration 1 successfully delivers an initial high-level system architecture for AIDAP:

- The Web Application reference architecture defines how the users interact with the system.
- The Microservices reference architecture outlines how core functionality is divided across server-side services.
- The three-tier deployment pattern establishes the physical layout of the system and enables cloud-based deployment.

However, on this stage, most architectural drivers are only partially implemented; as a result, future implementations will focus on:

- Improving performance tactics to better satisfy QA-1
- Defining detailed redundancy, monitoring, and failover approaches to complete QA-2 and CON-5
- Selecting specific security frameworks and mechanisms to fully address QA-4, CON-1,CRN-3.
- Establishing concrete integration services and communication protocols to fully satisfy CRN-1 and CON-2.

Therefore, the goal of establishing a general architectural framework has been achieved and the remaining gaps are now identified as targets for future development cycles.

ITERATION 2

Step 2 - Establish the Iteration Goal:

The goal of this iteration is to address general architecture concerns of identifying structures to support primary functionality.

UC-1: Always allows a student to ask academic inquiries, such as when their next exam is. The system will then understand the question using Natural Language Processing. It will further output the date and time from the school.

UC-4: Lecturers shall be able to post announcements by text or voice to their students.

UC-8: The maintainer shall be able to put out new AI models and API key configurations without downtime.

Step 3 - Choose One or more Layers to refine

The elements that we will refine in this iteration are modules located in the different layered defined by the reference architectures we used in the previous iteration. In order to support the functionality of the system, we require collaboration of components with these modules in these different layers.

Step 4 - Choose one ore more design concepts that satisfy the selected drivers

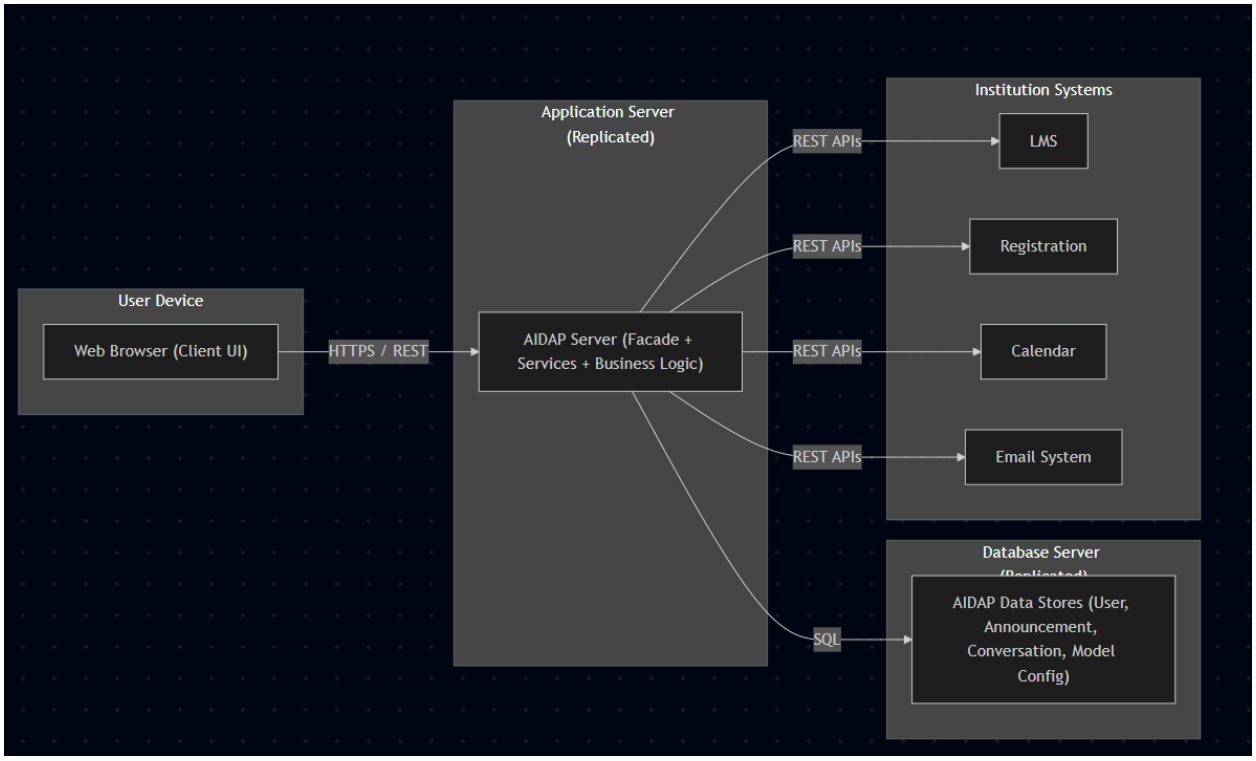
Design Decisions and Location	**Rationale**
Create Domain Model for our AIDAP	Creating an initial domain model for the system is used to clearly define our use cases in our early architecture, We need the domain model to map UC-1, UC-4, and UC-8 specifically to ensure each functional requirement is contained in a building block.
Introduce a Facade layer between Client and Server Modules	Based on our current use cases, since UC-1 and UC-4 requie several different modules that are between different layers, a facade provides a joint entry point, hiding the internal complexity of the system. This would in turn reduce coupling in the system and ensures consistent communication paths.
Utilize REST Communication through API Gateway	REST allows us to use a simple, standardized communication though a single gateway, satisfying security drivers by centralizing authentication and request validation before business logic is executed.
Identify Domain Objects that map to functional requirements	To ensure each primary use case is supported, UC-1, UC-4 and UC-8 must be clearly defined early in the architecture. Each distict functional element should be encapsulated in a Domain Object, ensuring structure for the modules refined in the iteration.

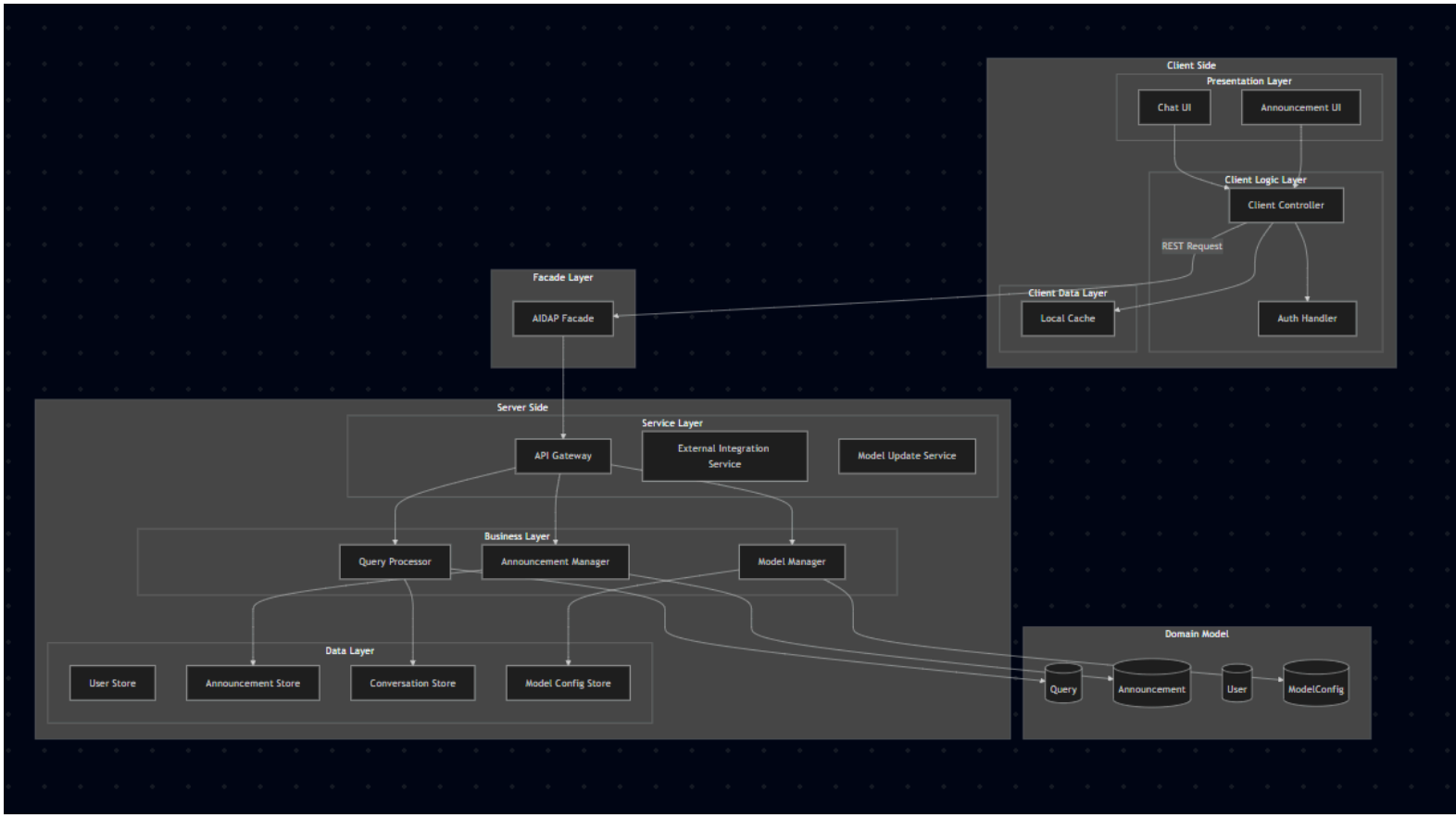
Step 5 - Instantiate Architectural Elements, Allocate responsibilities, define interfaces

Design Decision	**Rationale**

Create Domain model for AIDAP	The domain model clarifies system entities and supports early mapping of UC-1, UC-4 and UC-8, ensuring every functional requirement is represented by a corresponding building block
Introduce a Facade Layer between Client and Server Modules	UC-1 and UC-4 rely on multiple modules across layers. A facade provides a unified entry point, hides internal complexity, reduces coupling, and ensures consistent communication paths.
Utilize REST Communication Through an API Gateway	Ret offers simple, standardized communication. The API Gateway centralizes authentication, request validation, and routing, improving security and consistency before business logic is executed.
Identify Domain Objects That Map to Functional Requirements	Ensures proper support for UC-1, UC-4 and UC-8 by encapsulating each functional element into domain objects that form the basis of responsibilities and service boundaries.

Step 6 - Sketch views and record design decisions


Application Server
(Replicated)
Institution Systems
REST APIs
LMS
REST APIs
Registration
User Device
Web Browser (Client UI)
HTTPS / REST
AIDAP Server (Facade +
Services + Business Logic)
REST APIs
Calendar
REST APIs
Email System
Database Server
SQL
AIDAP Data Stores (User,
Announcement,
Conversation, Model
Config)


Client Side
Presentation Layer
Chat UI
Announcement UI
Client Logic Layer
Client Controller
REST Request
Client Data Layer
Local Cache
Auth Handler
Facade Layer
AIDAP Facade
Server Side
Service Layer
API Gateway
External Integration
Service
Model Update Service
Business Layer
Query Processor
Announcement Manager
Model Manager
Data Layer
User Store
Announcement Store
Conversation Store
Model Config Store
Domain Model
Query
Announcement
User
ModelConfig

Element	Responsibility
Chat UI	Displays the conversational interface, sends both staff and student questions to the backend, and displays the answers (UC-1)
Dashboard UI	Displays personalized dashboard that includes deadlines, academic information, and events pertinent to the user that is logged in (UC-3)
Client Controller	Screen flow is managed between both the chat and the dashboard, and forwarding of user actions from the UI to the API Gateway
Auth Handler	Start single sign on login, tracks user that is logged in, and connects identity information to client requests (UC-6)
Local Cache	Stores recent data temporarily on the client side to help improve performance, as well as help with limited connectivity
Logging	Records events and requests throughout the system that is important to allow behaviour to be analyzed and reviewed later (UC-7)
Security	Security checks that include academic data protection and access control.
Error Handling	Finds out and handles failures in a way that returns error information to either user or maintainer.
API Gateway	Serves as the main gateway for client requests, performing initial checks and directing each requests to the correct backend service
External Integration Service	Handles connections to the institutional platforms like the LMS, registration system, calendar, and emails ensuring that AIDAP’s internal data remains synchronized with the systems
Query Processor	Understands User questions, works with AI components and data sources, and returns reponses for academic or scheduling inquiries
Dashboard Manager	Gathers and aggregates the information required for a user's dashboard, including upcoming tasks, assignments and events.

Announcement Manager	Enables instructors and administrators to create and manage course or campus announcements that students can view and receive notifications for
User Data Store	Maintains user profiles, roles and preferences that are required for authentication and personalized behaviour
Conversation Store	Stores a history of interactions between the user and the assistant so the system can offer context-aware and personalized responses while still following privacy rules
Analytics Store	Saves combined usage and performance data that is used for system monitoring and generating reports.

Step 7 - Perform Analysis of Current Design and review Iteration Goal and Achievement of Design Purpose

The design choices made in iteration 2 includes:

- **Developing a Domain Model for AIDAP.**
- **Adding a Facade Layer between theclient and server components.**
- **Using REST-based communication via the API Gateway.**
- **Identifying th Domain objects that correspond to the selected functional requirements.**

Driver	**Status**	**How it is addressed**
UC-1, UC-8, and UC4 need to be represented in the domain model	Partial address	The domain model introduction, as well as explicit domain objects makes sure query handling, AI model updates, and course announcements all have dedicated concepts in the architecture. Though for the exact list of domain objects, their respective relationships, and their respective attribute
QA-5 - Modifiability (ease of update)	Partial address	The placement of a facade between the client and server helps reduce coupling which makes it easier to modify internal logic with no dependency. Future iterations need to refine the way domain objects are grouped into modules and how API versioning will be handled to satisfy this QA.
QA-4 - Security	Partial address	REST used through API Gateway where facade pattern

		centralizes the request validation and authentication before the business logic execution.
CON-1 - University privacy rules for academic data is followed	Partial address	Domain model identifies academic entities which is a prerequisite for defining sensitive data. Having these well devinfes domain objects and a facade givea a single consistent point from data to external system Translation in the AIDAP's internal models.
CRN-1/ CRN-3 - Consistency of integration and the secure handling of academic information	Partial address	Having well-defined domain objects and a facade gives a single, consistent point where data from external systems is translated into AIDAP's internal model and checked before use.

Iteration 2 focused on refining the domain model and establishing a clear facade boundary so that the key use cases are better supported and the system can grow safely.

- The Domain Model and Domain objects now give a clearer structure for handling queries and AI-related functionality.
- The combination of Facade/API Gateway and REST communication offers a unified access point for these operations and starts to tackle concerns around security and modifiability.

Overall, many of the drivers are partially met.